

Reto integrador — Plataforma cloud regulada: DevOps, CloudOps y diseño cloud-native

Documento único de enunciado para equipos del curso. Contiene **contexto**, **reglas (diseño vs implementación)**, **entregables**, **rúbrica** y **referencias**.

Índice

1. Objetivo del reto
2. Contexto de negocio: VitalTrace HealthNet
3. Escenario: territorio Colombia, on-premises y datos (incluye sección 3.5 — Microsoft Fabric)
4. Estrategia de migración nacional
5. Regla central: diseño frente a implementación · 5.2 Checklist núcleo · 5.3 Reporte de costos de la implementación
6. Qué significa DevOps y CloudOps en este reto
7. Nivel respecto a trabajos previos del programa
8. Requisitos transversales
8.1. CNCF Landscape · 8.2. Well-Architected · 8.3. Regiones e infraestructura en Colombia · 8.4. Diagramas
9. Reto por bloques técnicos
10. Alcance explícito y fuera de alcance
11. Tareas del equipo
12. Entregables
13. Criterios de evaluación (rúbrica)
14. Reglas generales

15. Mensaje final

1. Objetivo del reto

Diseñar y sustentar una **plataforma cloud** para operación nacional de salud digital (**VitalTrace HealthNet**), con **infraestructura como código, pipelines, GitOps, seguridad, observabilidad y monitoreo operativo, FinOps** y **migración desde on-premises**, más una **propuesta de modernización del dato** centrada en **Microsoft Fabric** (o equivalente justificado si el proveedor principal no es Azure), alineada al **Well-Architected Framework** del proveedor elegido (**Azure, AWS o GCP**) y al ecosistema **CNCF** donde aplique.

- **No** se evalúa desarrollo de la **lógica de negocio** clínica (los equipos de producto entregan cargas **containerizadas**).
- **Sí** se evalúa todo lo necesario para **operar esa plataforma en la nube**: diseño, automatización, cadena de suministro, continuidad, **monitoreo, modernización del dato** (Microsoft Fabric o equivalente) y cultura DevOps/CloudOps.

Supuesto pedagógico: dominio de **Kubernetes**; el foco no es explicar Pods básicos, sino **decisiones de plataforma** (red, identidades, políticas, costos, liberación).

2. Contexto de negocio: VitalTrace HealthNet

VitalTrace HealthNet S.A.S. es un operador de **plataforma nacional** de salud digital en Colombia: integra IPS públicas y privadas, laboratorios, aseguradoras y programas de salud pública. Busca **mejorar interoperabilidad y continuidad asistencial**, reducir **fragmentación** y tiempos de espera, y fortalecer **trazabilidad y auditoría**.

La compañía opera bajo marco regulatorio estricto (protección de datos personales, trazabilidad sobre información sensible de salud, **retención legal prolongada**, auditorías y **preferencia de residencia de datos** alineada al país). En el diseño deben **explicitar** regiones de nube, cifrado, gestión de llaves y gobernanza.

3. Escenario: territorio Colombia, on-premises y datos

3.1. Territorio y red

El país se articula por las **32 capitales departamentales y Bogotá, D.C.** (33 cabeceras). VitalTrace **no** depende de un solo centro de datos: hay una **sede nacional principal** (ciudad **justificada** por el equipo) y **puntos de presencia lógicos**:

- **Concentradores regionales** (p. ej. **6 a 10** regiones que agrupan capitales), cada uno con perimetral (firewall, VPN, proxy) y enlaces **VPN sitio a sitio** o **MPLS** hacia la sede central.
- **Capitales** como borde: muchas IPS se conectan a los concentradores. No deben modelar cada IPS; sí **patrón de conectividad, latencias heterogéneas, ancho de banda asimétrico** y **partición regional** en el diseño.

3.2. On-premises (VMware)

En la sede principal (y en algunos concentradores grandes), tres dominios lógicos:

- **Datos:** PostgreSQL (prod / QA / dev), Redis (colas, caché de integración, reintentos hacia sistemas legados y bus de interoperabilidad).
- **Aplicaciones:** VMs con Docker; plataforma modular (p. ej. Spring Boot) por contextos: admisión, autorizaciones, laboratorio, facturación interoperable (RIPS/FHIR u otros mensajes — solo como **carga**), portal de resultados, orquestación entre prestadores.
- **Plataforma y objeto:** documentos clínicos, adjuntos, informes, **imagenología**; monitoreo clásico (Icinga); Nginx/Apache.

3.3. Datos: ambición nacional vs inventario certificado

En teoría, un país acumularía volúmenes enormes. En la práctica — **frecuente en grandes programas en Colombia** — el **inventario no es único ni confiable**: rotación de prestadores, consolidaciones fallidas, backups incompletos, duplicados, migraciones de IPS sin cierre de ficheros, **pérdida de metadatos** y, en algunos casos, **pérdida de objetos** (medios obsoletos, llaves mal rotadas). Por ello el **volumen certificado y gobernable** para el proyecto a la nube es **menor** que la ambición de un **"país completo en petabytes"**.

El equipo debe trabajar en **dos capas**:

1. **Ambición / deuda (diseño + FinOps extrapolado):** qué ocurriría si se cerraran brechas de inventario y creciera el ecosistema (referencia de órdenes de magnitud mayores).
2. **Contrato de migración actual (diseño + costos y oleadas):** cifras **firmadas** tras auditoría: lo que **sí** se mueve en oleadas. Esas cifras alimentan **FinOps**, ventanas y pruebas de restauración **documentadas**.

Clase de dato	Referencia ambición país (diseño / riesgo futuro)	Volumen certificado hoy (on-prem hacia nube; $\pm 20\%$ si documentan supuestos)
Objeto caliente	Orden de petabytes si todo estuviera unificado	~120–350 TB útiles certificados (huérfanos/duplicados fuera de inventario)
Imagenología	Multi-PB teórico	~0,4–1,2 PB accesible en filer/SAN; bloques congelados o ilegibles fuera del contrato (solo riesgo)
Archivo legal/frío	PB adicionales en teoría	~80–250 TB con custodia válida; cinta sin índice = deuda (opcional: diseño forense, no migración masiva)
Ingesta pico	Decenas de TB/día en teoría	~2–8 TB/día pico certificado en bus de integración (últimos 12 meses)

En **laboratorio** no se aprovisionan esos volúmenes: se **implementan** políticas y muestras; se **documentan** costos y oleadas para **ambas** capas.

3.4. Dolores organizacionales y visión

- Brecha territorial (calidad de enlace: Orinoquía, Amazonía, **Pacífico**, grandes urbes); SLOs o **edge** por región.
- Costo: almacenamiento y **egreso** pueden superar cómputo; FinOps por componente y región.
- **RTO/RPO** por **capacidad crítica** (p. ej. autorizaciones vs archivo no urgente), no un solo número genérico.
- Migración con convivencia entre on-prem y nube, **cortes** y **rollback** por oleadas.
- Política como código y auditoría centralizada en la nube.

Visión: plataforma **cloud-native multi-región**, **Git** como fuente de verdad para infraestructura y políticas, **postura de cero confianza**, identidades de carga de trabajo, **observabilidad y monitoreo** con **SLOs**, alertas accionables e

incidentes, **unificación progresiva del dato** para analítica y gobernanza (véase **Bloque 9 — Microsoft Fabric** en la sección 9 de este documento), cadena de suministro verificable, **CI/CD y GitOps** con seguridad, costo y cumplimiento; **colaboración y mejora continua** con el negocio.

3.5. Modernización del dato con Microsoft Fabric (referencia obligatoria en diseño)

VitalTrace necesita **ordenar, gobernar y explotar** el dato que **sí** tiene certificado (y preparar la llegada del resto cuando se cierre deuda de inventario). Se exige una **propuesta explícita** usando **Microsoft Fabric** como **línea base** cuando el proveedor de nube principal sea **Azure**:

- **Qué deben plantear (diseño mínimo):** *workspaces* y separación de entornos (desarrollo / analítica operacional), **Lakehouse** y/o **Warehouse** (justificar cuándo usar cada uno), **OneLake**, accesos a datos en **Azure Data Lake Storage** (objeto certificado), **Shortcuts** o ingesta desde el bus de interoperabilidad y réplicas de lectura, **pipelines** de ingeniería de datos (calidad, deduplicación, linaje), **capas** tipo bronce/plata/oro o equivalente defendible.
- **Gobernanza y cumplimiento:** catálogo de datos (p. ej. integración con **Microsoft Purview** donde aplique), clasificación de datos sensibles, **mínimos privilegios** en Fabric, auditoría de consultas, **no** mezclar analítica abierta con datos clínicos sin controles (zonas de confianza, *private link*, *VNet* integration según documentación vigente).
- **Operación y monitoreo del dato:** cómo se **supervisa** salud de *pipelines*, retrasos de ingesta, fallos de *refresh* y costo de capacidades (*capacities*); alertas hacia el mismo modelo de operación que el resto de la plataforma.
- **Si el proveedor principal es AWS o GCP:** deben documentar un **equivalente** (p. ej. combinación de lago + almacén + orquestación + catálogo del ecosistema de ese proveedor) y un **ADR** "Fabric vs stack elegido" con trade-offs; el nivel de detalle debe ser **comparable** al de un equipo que eligió Azure con Fabric.

Implementación en laboratorio (núcleo): basta con **diseño detallado + diagrama(s)** y, si pueden, **recursos mínimos** (p. ej. *workspace* y un *lakehouse* de prueba sin datos reales). **Ampliación:** *pipeline* real de muestra, *notebook* o políticas de acceso implementadas en código/IaC donde exista proveedor Terraform/API.

4. Estrategia de migración nacional

El reto **no** termina en “levantar un clúster Kubernetes”. Deben plantear **cómo** migra el país mientras sigue operando on-premises. Mínimo: **documentación + diagramas**; donde sea viable, **automatización** en IaC/pipelines.

1. **Inventario y clasificación:** crítico en línea (autorizaciones, mensajería), analítico, archivo; dependencias PostgreSQL, Redis, bus, **objeto**.
2. **Oleadas (mínimo 3 nombradas):** p. ej. (A) fundaciones en nube + identidad + conectividad híbrida; (B) integración y APIs sin mover todo el histórico; (C) traslado caliente → templado → frío con **checksums** y pruebas de restauración.
3. **Red y gravedad de datos:** evitar mover el mismo byte dos veces; **transferencia offline** (p. ej. dispositivos tipo *Data Box*), enlaces dedicados o VPN ampliada **justificada**; costo de **replicación inter-región** e ingreso/egreso. Mencionar el **vacío** entre inventario certificado y deuda histórica.
4. **Convivencia híbrida:** DNS global/privado, ruteo por región, **split horizon**, consumo desde concentradores y capitales sin romper cumplimiento.
5. **Corte y rollback:** criterios medibles para tráfico productivo; **vuelta atrás** por oleada si fallan SLOs o integridad.
6. **Operación dual:** observabilidad on-prem + nube (al menos como diseño); claridad de **responsabilidades** (enlace regional vs fallo de clúster).
7. **ADR obligatorio** solo sobre **estrategia de migración** (oleadas, riesgos, alternativas descartadas).

5. Regla central: diseño frente a implementación

La narrativa es **nacional y ambiciosa**; la **implementación en la suscripción de laboratorio** es **acotada** por tiempo y presupuesto académico.

Aspecto	Diseño (documentación, diagramas, ADRs)	Implementación (repositorio, nube, pipelines)
Meta	Demostrar comprensión del macro: riesgos, trade-offs, costos, oleadas, regiones, Well-Architected, CNCF.	Demostrar ejecución: IaC modular, GitOps, <i>gates</i> , entorno dev/staging creíble.

Aspecto	Diseño (documentación, diagramas, ADRs)	Implementación (repositorio, nube, pipelines)
Alcance	Todo lo del enunciado debe estar razonado y trazado .	Subconjunto sin petabytes reales, sin enlazar las 33 capitales en lab, sin multiregión completa salvo que lo documenten como decisión de equipo.
Criterio	Coherencia y defensa ante panel; supuestos explícitos.	Código revisable, pipelines verdes en lo prometido, evidencia en clúster para lo declarado en lab.

Regla de oro: lo que quede solo en diseño debe llevar la etiqueta "**Diseño: no desplegado en laboratorio por ...**" y, aun así, **política, diagrama o costo estimado**, para que el evaluador no busque recursos inexistentes.

5.1. Núcleo (aprobación sólida) vs ampliación (excelencia)

Ámbito	Núcleo	Ampliación
ADRs	Mínimo 6 (migración, regiones/edge, comparativa CNCF, y otros temas de arquitectura).	8 ADRs en total (+2 con malla, FinOps fino, observabilidad avanzada, etc.).
Políticas en la nube (IaC/CI)	≥5 definidas en código o plantillas y referenciadas en CI.	≥10 documentadas y el exceso sobre 5 implementado o en PR hacia staging.
Matriz Well-Architected	≥2 filas por pilar (decisión → evidencia o " diseño: pendiente en lab ").	≥3 filas por pilar y revisión de riesgos más densa.
Diagramas	D1–D10, D13 (incluye D8 observabilidad y monitoreo; D7 con zona analítica Fabric o equivalente).	D11, D12 (STRIDE y CNCF ampliado).
Multi-región / DR	Diseño + al menos una región implementada o secundaria simulada (IaC con <i>feature flag</i> o comentado).	Segunda región con recursos mínimos reales y conmutación documentada.
CNCF	Mapa + 2 capacidades nuevas documentadas ; ≥1 con evidencia ligera en repo.	Ambas integradas en pipeline o clúster.

El **macro** queda dicho en arquitectura y en los diagramas de núcleo; la **ampliación** premia el pulido sin castigar a quien priorizó bien el núcleo.

5.2. Checklist rápido (núcleo) — autocontrol antes de entregar

Los equipos pueden usar esta lista para verificar que no falta nada **obligatorio en mínimo** antes de la sustentación:

#	Requisito	Dónde se exige en este documento
1	≥6 ADRs que cubran al menos: migración y oleadas, regiones y puntos de borde (edge), comparativa CNCF, Microsoft Fabric o equivalente de datos , y dos decisiones más coherentes con el repositorio (red, identidades, GitOps, <i>mesh</i> , observabilidad, etc.)	secciones 5.1, 4, 8.1 y 8.3; Bloque 9 (sección 9)
2	Diagramas D1-D10 y D13 con índice, leyenda, versión/fecha y la leyenda " diseño solamente " donde no haya recurso en lab	secciones 8.4 y 14
3	D7 con volumen certificado vs deuda y zona analítica (Fabric o equivalente)	secciones 3.3 y 8.4; Bloque 9 (sección 9)
4	D8 con monitoreo operativo (infra, plataforma, híbrido, sintético si aplica) y rutas de alerta	sección 8.4; Bloque 6 (sección 9)
5	≥5 políticas en la nube en laC/CI + ≥10 políticas documentadas en conjunto	Bloque 1 (sección 9)
6	Kubernetes privado, GitOps (<i>App of Apps</i>), 2 pipelines , OIDC a la nube donde sea viable	Bloques 3 y 7 (sección 9)
7	Mapa CNCF (tabla en documentación en núcleo), 2 capacidades nuevas justificadas, ADR comparativo	sección 8.1
8	Matriz Well-Architected (≥2 filas/pilar) + ≥5 riesgos en la revisión Well-Architected simulada	sección 8.2
9	FinOps y reporte de costos de la implementación (sección 5.3): tabla por componente con volumen certificado ; Infracost (o similar) con umbral que bloquee <i>merge</i> ; partida presupuestal u orden de	sección 5.3; Bloques 5 y 9 (sección 9)

#	Requisito	Dónde se exige en este documento
	magnitud para Microsoft Fabric (<i>capacities</i>) o equivalente analítico	
10	Backup / restore demostrable en lab; deriva con efecto en <i>pipeline</i>	Bloque 5 (sección 9)
11	SLOs , alertas, <i>runbooks</i> , prueba de carga a endpoint de plataforma	Bloque 6 (sección 9)
12	STRIDE (1–2 páginas); el diagrama D11 (ampliación) puede resumir el mismo modelo de amenazas de forma gráfica	Bloque 4 (sección 8.4)
13	Plan de migración nacional (documento o capítulo: oleadas, <i>rollback</i> , híbrido) alineado al D10	sección 4
14	Regiones con nombres y enlaces oficiales del proveedor	sección 8.3
15	Cinco capitales reales mencionadas en diseño de red	Bloque 2 (sección 9)
16	Profundización : una opción del Bloque 8 (sección 9)	sección 9
17	Reflexión IA (1–2 páginas)	Bloque 10 (sección 9)
18	Demo (máx. 15 min) y presentación	sección 11

Ampliación (nota alta): diagramas **D11 y D12**, **8 ADRs**, **≥3** filas por pilar en Well-Architected, **≥10** políticas materializadas en código donde aplique, segunda región con evidencia, recursos de prueba de Fabric u orquestación analítica, etc. (secciones 5.1 y 10).

5.3. Reporte de costos de la implementación

El **reporte de costos** es **obligatorio en núcleo**: debe mostrar **cómo queda el costo** de lo que el equipo **sí implementó** en laboratorio y, en paralelo, **cómo se proyecta** el costo del diseño nacional usando el **volumen certificado** (sección 3.3). Sin ese contraste, el evaluador no puede verificar coherencia entre repositorio, diagramas y FinOps.

Contenido mínimo del reporte (un capítulo del documento de arquitectura o archivo dedicado, p. ej. `docs/costos-implementacion.md`):

Bloque del reporte	Qué debe incluir
1. Alcance	Región(es), moneda, fecha de la cotización o export de calculadora; lista de recursos desplegados en lab (nombre lógico, tipo, SKU si aplica).
2. Costo mensual de la implementación (lab)	Tabla: recurso o componente → unidad (horas, GB-mes, solicitudes, etc.) → precio unitario de referencia (con enlace a calculadora o lista de precios oficial del proveedor) → subtotal mensual. Total mensual estimado del laboratorio y, si aplica, captura o enlace a salida de Infracost alineada al mismo <i>commit</i> o etiqueta.
3. Diseño a escala (no desplegado o parcial)	Tabla por componente (cómputo AKS/EKS/GKE, almacenamiento por clase , red, observabilidad, PaaS, capacidad analítica Fabric o equivalente) usando volumen certificado ; marcar con " diseño solamente " lo que no esté facturado en la suscripción de lab.
4. Sensibilidad	Variación ±20% en al menos dos variables (p. ej. TB almacenados, réplicas, egreso mensual) y efecto en el total.
5. Supuestos y riesgo de precio	Qué queda fuera del cálculo (reservas, descuentos educativos, cuotas); advertencia de que los precios cambian y la fuente es la documentación oficial.

Criterio de calidad: un lector debe poder responder en una página: **¿cuánto cuesta al mes lo desplegado?** y **¿en qué orden de magnitud quedaría el mes típico del diseño** con datos certificados, sin mezclar ambas cifras sin etiquetar.

6. Qué significa DevOps y CloudOps en este reto

No basta con Terraform y GitHub Actions. Las dimensiones siguientes deben verse en diseño; la **implementación** sigue los umbrales del **núcleo** (sección 5).

Dimensión	Qué demostrar
Plan y colaboración	Ramas, PR, revisiones, <i>ownership</i> ; SLI/SLO a nivel plataforma con "producto".
Código y artefactos	Repos claros, versionado, plantillas; imágenes y <i>charts</i> con trazabilidad (<i>digest</i> , SBOM si aplica).
Build y prueba	Pipelines que validan IaC y políticas; <i>gates</i> que bloquean promoción defectuosa.
Liberación y despliegue	GitOps y/o CD; promoción dev → staging → prod (o <i>prod</i> simulado); <i>rollback</i> documentado.

Dimensión	Qué demostrar
Operación, observabilidad y monitoreo	Métricas, logs y trazas; monitoreo sintético o <i>healthchecks</i> hacia la superficie de plataforma; monitoreo de infraestructura (nodos, disco, red, plano de control del clúster); alertas con <i>runbooks</i> y rutas de escalamiento; prueba de carga sobre <i>ingress</i> u otro endpoint de plataforma, no sobre lógica clínica propia.
Seguridad y cumplimiento	<i>Shift-left</i> , secretos, identidades, segmentación; ejercicio breve de incidente simulado.
FinOps	Presupuestos y costo en pipeline; modelo para volumen certificado y proyección de ambición país; clases de almacenamiento, replicación, egreso.
Mejora continua	Qué aprendieron de la deriva de infraestructura, fallos de pipeline o <i>chaos</i> ; retrospectiva breve.

CloudOps incluye: gobernanza de nube, operación de planos de datos y control, parches del clúster, *backup/restore*, DR, capacidad y comunicación con seguridad, finanzas y producto.

7. Nivel respecto a trabajos previos del programa

Pueden **reutilizar y extender** lo ya logrado (multi-región en Azure, AKS, ACR, Traffic Manager, GitHub Actions, GitOps con Argo CD, Helm, Terraform, *service mesh* Linkerd, mTLS, Prometheus/Grafana/Loki/Tempo, Checkov/Trivy, Infracost, clúster privado, Bastion, Locust, etc.), según la publicación en LinkedIn y el portafolio DevOps 2026-1.

Se espera **mayor madurez de diseño** y **ciclo DevOps/CloudOps explícito**: plataforma **operable**, no solo una lista de herramientas encendidas.

8. Requisitos transversales

8.1. CNCF Landscape

Consultar el **CNCF Cloud Native Interactive Landscape** y justificar el *stack*.

1. **Mapa:** componentes principales (CI, GitOps, *registry*, *mesh* o alternativa, observabilidad, política, almacenamiento, mensajería si aplica) y su **categoría** en el *landscape*.
2. **Dos proyectos o capacidades "nuevas"** para el equipo (respecto al proyecto previo) o uso sustancialmente distinto de un proyecto *Graduated*;

documentar **por qué** y **qué riesgo** mitigan.

3. **Comparativa** en al menos **una** categoría clave (p. ej. GitOps, *mesh*, admisión): **dos** opciones del *landscape* → decisión en **ADR**.

El *landscape* es **catálogo**, no acumulación de logos: prima la **coherencia**.

En **núcleo**, el mapa CNCF puede presentarse como **tabla** en la documentación; el diagrama **D12** es la versión gráfica completa y corresponde a la **ampliación**.

8.2. Well-Architected Framework

Acoplarse al marco del proveedor (**Azure**, **AWS** o **GCP**):

Proveedor	Documentación oficial
Azure	Azure Well-Architected Framework
AWS	AWS Well-Architected Framework
Google Cloud	Google Cloud Architecture Framework

- **Matriz:** por cada **pilar**, filas con decisión/control, validación o implementación (IaC, política, pipeline, *runbook*), evidencia en repo o la etiqueta "**diseño: pendiente en lab**". Umbrales: **núcleo** ≥ 2 filas/pilar; **ampliación** ≥ 3 filas/pilar.
- **Well-Architected Review (simulado):** ≥ 5 riesgos con severidad y mitigación (núcleo); ampliación: más riesgos o tolerancia residual.

Si mezclan términos entre proveedores, deben **mapearlos** explícitamente.

8.3. Regiones e infraestructura en Colombia

No inventen nombres de región: verificar en la documentación **oficial** del proveedor al momento del proyecto y **citar** enlaces.

Proveedor	Orientación (verificar siempre en fuentes oficiales)	Implicación
Azure	Infraestructura de nube en Colombia (p. ej. Bogotá y alrededores); confirmar nombre programático , zonas de disponibilidad y servicios en Geografías de Azure y Lista de regiones .	Carga primaria en Colombia si el servicio lo permite; DR según emparejamiento y cumplimiento.
AWS	Suele no haber una región completa solo en Colombia (varias AZ bajo un mismo código); sí puntos de borde (p. ej. Bogotá	Región ancla (p. ej. sa-east-1) + edge en

Proveedor	Orientación (verificar siempre en fuentes oficiales)	Implicación
	para CDN). Ver <u>Infraestructura global de AWS</u> . <u>Local Zones</u> u otras extensiones según catálogo.	Colombia; residencia de datos vs latencia.
GCP	Regiones suramericanas habituales: Santiago (<code>southamerica-west1</code>), São Paulo (<code>southamerica-east1</code>), etc. <u>Ubicaciones</u> . <u>Selector de región</u> .	Multi-región o multi-zona justificada desde capitales.

ADR obligatorio sobre elección de regiones / *edges*, impacto en **RTO/RPO**, costo de red y cumplimiento.

8.4. Catálogo de diagramas

Núcleo: D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D13 (el **D8** cubre observabilidad y monitoreo operativo). **Ampliación:** D11, D12.

Formato libre (Draw.io, Excalidraw, Lucidchart, Mermaid exportado a PDF/SVG). Cada uno: **leyenda**, datos **ficticios** si aplica, **versión y fecha**.

ID	Tema	Contenido mínimo
D1	Contexto (C4)	VitalTrace, prestadores, paciente como actor, regulador, nube elegida.
D2	Territorio Colombia	Capitales / concentradores on-prem, VPN/MPLS, flujo hacia la nube (nombres de ciudades bastan).
D3	Red híbrida	<i>Hub-spoke</i> o VWAN / TGW, subredes, firewalls, DNS privado, Private Link, rutas a on-prem.
D4	Kubernetes	Plano de control gestionado, nodos, CNI, <i>ingress</i> , <i>mesh</i> o sustituto, <i>namespaces</i> críticos.
D5	Identidades	SSO humanos, OIDC en CI, identidad de carga de trabajo hacia KMS/secretos; <i>break-glass</i> .
D6	GitOps y pipelines	Repos → CI → artefactos → GitOps → clúster; <i>gates</i> de seguridad y costo.
D7	Datos operacionales y analíticos	Caliente/templado/frío, replicación, <i>backup</i> , KMS, gravedad de datos; certificado vs deuda/ambición; zona Microsoft Fabric (Lakehouse / <i>OneLake</i> / <i>pipelines</i>) o equivalente en otro proveedor, y flujos desde bus/objeto/PostgreSQL.

ID	Tema	Contenido mínimo
D8	Observabilidad y monitoreo	Agentes y recolectores; métricas, logs y trazas; <i>dashboards</i> por audiencia (NOC/SRE vs liderazgo); alertas enlazadas a <i>runbooks</i> ; comprobaciones de disponibilidad (sintéticas o <i>blackbox</i>) hacia APIs de plataforma y, si aplica, túneles VPN; correlación alerta / log / traza.
D9	DR	Primaria/secundaria o activo-pasivo; conmutación; RTO/RPO en el dibujo.
D10	Migración	Línea de tiempo u oleadas (tipo Gantt): prerequisites, cortes, <i>rollback</i> .
D11	STRIDE	Flujo de datos y mitigaciones en plataforma.
D12	CNCF	Su mapa frente al <u>Landscape</u> .
D13	Well-Architected	Vista pilares → componentes principales.

Reglas: (1) Si unen D2 y D3, mantener legibilidad y anexos numerados. (2) En D7, **ambas** capas: certificado para migrar vs deuda/ambición (caja punteada). (3) Recursos no desplegados: leyenda "**diseño solamente**".

9. Reto por bloques técnicos

Bloque 1 — Gobernanza y *landing zone*

- Jerarquía (management groups / carpetas / OUs): **plataforma, cargas de trabajo, seguridad y auditoría, sandbox**.
- Políticas en la nube: **≥10 documentadas; núcleo ≥5** en IaC/CI; ampliación: resto materializado. Admisión en clúster (Kyverno, Gatekeeper/OPA u otra) **versionada** vía GitOps (*núcleo*: patrón mínimo; *ampliación*: más cobertura).
- RBAC de mínimo privilegio; **OIDC** desde CI a la nube; *break-glass* documentado.

Bloque 2 — Red y conectividad

- *Hub-spoke* o VWAN / Transit Gateway; conectividad entre on-prem y nube con alta disponibilidad descrita; DNS privado; Private Link hacia PaaS sensibles.
- **Al menos cinco capitales** reales como origen o agregación de tráfico; efectos en latencia, MTU, pérdida o enrutamiento (cualitativo + supuestos

numéricos si aplica).

- Acceso a equipos **sin** SSH público a nodos (Bastion, SSO, acceso *just-in-time* — diseño; implementación según presupuesto).

Bloque 3 — Kubernetes y CNCF

- **AKS / EKS / GKE** (elegir y justificar): clúster **privado** (o equivalente), identidad de carga de trabajo del proveedor.
- GitOps (Argo CD / Flux), **App of Apps** (o equivalente) para *addons* y cargas de referencia.
- *Service mesh* recomendado; si no: **ADR** con alternativa (mTLS gestionado, *NetworkPolicies*, observabilidad).

Bloque 4 — Seguridad y cadena de suministro

- Secretos (KMS, CSI, rotación o integración); escaneo de IaC e imágenes en CI; firma o política de admisión de imágenes si es viable.
- WAF / mitigación DDoS como plantilla o código.
- **STRIDE** corto (1–2 páginas) en plataforma y *pipeline*.

Bloque 5 — Confiabilidad, SRE y CloudOps

- **RTO/RPO** con números; **mínimo dos perfiles** (p. ej. ruta crítica de autorizaciones vs consulta de archivo/imagenología), justificados con el mapa de Colombia.
- *Backup y restore* demostrables en laboratorio; para masivos: políticas y prueba de restauración **documentada** (puede ser muestra).
- **Deriva** de infraestructura con consecuencia en el *pipeline* (fallo o acción simulada).
- FinOps: presupuestos/alertas como código; **Infracost** (o similar) con **umbral** que bloquee *merges* si se supera el tope acordado (definir y justificar el tope). Tabla de costos: cómputo, almacenamiento por clase, red/egreso, observabilidad, usando **volumen certificado**; ampliación: proyección de **ambición país**. Incluir **línea o orden de magnitud** de costo para la **capacidad analítica** (p. ej. *Microsoft Fabric capacities* o equivalente en AWS/GCP) enlazada a la propuesta del **Bloque 9**. El detalle consolidado va en el **reporte de costos de la implementación** (sección 5.3).

Bloque 6 — Observabilidad y monitoreo operativo

Observabilidad (telemetría unificada):

- Métricas, logs y trazas (idealmente **OpenTelemetry** documentado o en uso).
- SLOs de plataforma, alertas y *runbooks* (p. ej. una página por alerta crítica).
- Prueba de carga (p. ej. Locust u otra herramienta del ecosistema) contra un **endpoint de plataforma**, con interpretación.

Monitoreo (disponibilidad y salud end-to-end): distinto de "solo tener Grafana"; deben definir **qué** se vigila y **cómo** reacciona el equipo:

- **Infraestructura y red:** nodos, CPU/memoria/disco, estado del CNI, balanceadores, **salud de conectividad híbrida** (p. ej. túneles VPN o enlaces críticos) como **checks** o métricas con umbrales.
- **Plataforma:** sincronización GitOps, fallos de *pull* de imágenes, certificados TLS próximos a vencer, colas de admisión.
- **Seguridad operativa:** correlación de alertas de escaneo o runtime (p. ej. Falco) con flujo de incidente.
- **Sintético / externo:** al menos **un** flujo de comprobación periódica hacia una URL o API de **plataforma** (diseño obligatorio; implementación en núcleo si el presupuesto lo permite).
- **Rutas de alerta:** quién recibe qué severidad (correo, chat, *ticketing* simulado) y tiempos de respuesta objetivo.

Bloque 7 — CI/CD, GitOps y gestión del cambio

- **Dos pipelines** con responsabilidades distintas, p. ej.: (1) fundaciones / IaC: *fmt*, *validate*, *lint*, escaneo, costo, política, *plan/apply* controlado; (2) clúster / GitOps / *release*: promoción por ambientes, *addons*, cargas de referencia, *rollback* o mitigación.
- Criterios de aprobación en PR para cambios que afectan **producción simulada**.

Bloque 8 — Profundización (elegir una)

- *Progressive delivery* de plataforma (Flagger / Argo Rollouts sobre un componente de plataforma), o

- *Chaos* controlado en no productivo (Chaos Mesh / Litmus / otro) con hipótesis y métricas de recuperación, o
- IDP mínimo (p. ej. Backstage) con plantillas Terraform/Helm aprobadas y trazabilidad.

Bloque 9 — Microsoft Fabric y mejora del uso del dato

Deben **proponer** cómo VitalTrace **aprovecha y ordena** el inventario **certificado** (y las fuentes operativas) mediante una **plataforma analítica unificada**, usando **Microsoft Fabric** como referencia principal si trabajan en **Azure**:

- **Ingesta y modelado:** desde almacenamiento objeto certificado, eventos del bus, réplicas de lectura o exportaciones controladas desde PostgreSQL; esquema en capas (bronce/plata/oro o equivalente); reglas de **calidad** y **deduplicación** alineadas al problema de inventario incompleto.
- **Arquitectura lógica:** *workspaces, capacities*, Lakehouse vs Warehouse, *OneLake, Shortcuts*, orquestación de *pipelines*; **costo** de *capacity* y estrategia de **entornos** (no mezclar desarrollo libre con datos sensibles sin controles).
- **Gobernanza:** linaje, catálogo, etiquetado de sensibilidad, accesos por rol (médico analista vs ingeniero de plataforma); integración con **Purview** u otra herramienta de catálogo según el catálogo oficial de Microsoft.
- **Monitoreo del dato:** métricas de retraso de *pipeline*, fallos de *refresh*, calidad rechazada; alertas hacia el mismo proceso operativo del bloque 6.
- **Proveedor AWS o GCP:** stack equivalente documentado al mismo nivel + **ADR** comparando con Fabric o explicando exclusión.

Entregables de este bloque: sección en el documento de arquitectura + **D7 actualizado** (o diagrama anexo enlazado) mostrando flujo **operacional** → **zona analítica**; en **núcleo** basta diseño y diagrama; **ampliación:** artefactos o recursos reales de prueba en la nube.

Bloque 10 — Inteligencia artificial en el proyecto

Se espera reflexión sobre cómo la **IA** (asistentes en IDE, chat, etc.) acorta ciclos en **diseño, planificación y estructura del repositorio**, sin sustituir verificación contra documentación oficial.

En qué fases suele ayudar más: borradores de ADR, riesgos Well-Architected, WBS y oleadas, esqueleto de carpetas y **README**, snippets de Terraform/Helm,

borradores Mermaid, consultas PromQL/LogQL, plantillas de *runbook*.

Donde no sustituye al ingeniero: nombres de regiones, SKU, cuotas y precios (solo fuentes oficiales); *trade-offs* sin contexto; `terraform apply` o cambios en prod simulada sin revisión humana y CI verde; cifras de inventario incoherentes con el **volumen certificado**.

Entregable: texto de **máx. 1–2 páginas** (o sección en `README` o `docs/ia-usage.md`) que responda: (1) qué aceleraron con IA; (2) cómo evitaron errores; (3) qué no delegaron y por qué; (4) **un** ejemplo de *prompt* o flujo (sin secretos ni datos reales de pacientes). La valoración es por **uso consciente y honesto**, no por volumen de uso.

10. Alcance explícito y fuera de alcance

En alcance	Fuera de alcance
Todo el sistema descrito en este documento, con separación diseño / implementación e inventario certificado vs deuda	Lógica de negocio clínica, modelado de datos sensibles, diseño UX del portal paciente
Núcleo: <i>landing zone</i> razonable, ≥5 políticas en código en la nube, identidades, red híbrida (diseño), Kubernetes, GitOps, CI/CD, observabilidad y monitoreo (bloque 6 + diagrama D8), propuesta Microsoft Fabric (o equivalente + ADR) en documento y D7 , <i>backup/restore</i> en lab, FinOps con volumen certificado y reporte de costos de la implementación (sección 5.3), SRE básico, mapa CNCF + 2 novedades, migración por oleadas documentada, Well-Architected (≥2 filas/pilar), diagramas de núcleo D1–D10 y D13 , regiones citadas con fuente	—
Ampliación: hasta ≥10 políticas en código, 8 ADRs, diagramas D11/D12 , ≥3 filas/pilar en WAF, segunda región con evidencia, <i>pipelines</i> o <i>workspace</i> Fabric de prueba, etc.	—
Helm/manifiestos de plataforma , <i>charts</i> de referencia o demos mínimos para validar <i>ingress</i> , políticas, SLO o carga	Certificación legal ante regulador
Flujo Git, PR, criterios de aprobación para <i>prod</i> simulada	—

11. Tareas del equipo

1. **Diseño y arquitectura:** documento maestro + **ADRs** (6 en núcleo, 8 en ampliación) incluyendo comparativa CNCF, migración nacional, regiones/*edge* y, cuando aplique, **Fabric vs alternativa de datos**.
 2. **Diagramas** según sección 8.4 (núcleo **D1–D10 y D13**; ampliación **D11 y D12**).
 3. **Matriz Well-Architected** y revisión de riesgos (umbrales sección 8.2).
 4. **Implementación:** IaC modular, estado remoto, GitOps, *pipelines*, *gates*, evidencias CNCF.
 5. **Operación:** SLOs, alertas, *runbooks*, **monitoreo** acorde al bloque 6, *restore* o *chaos* según bloque 8.
 6. **Costos:** modelo con supuestos y sensibilidad ($\pm 20\%$); volumen certificado + proyección de ambición; **reporte de costos de la implementación** (sección 5.3) con total mensual de lab y tablas separadas del diseño a escala.
 7. **Sustentación y demo** (vídeo o en vivo, máx. 15 min): Git → *pipelines* → GitOps → evidencia en clúster y observabilidad.
 8. **Propuesta de dato con Microsoft Fabric** (bloque 9): sección en documentación + **D7**; ampliación: recursos de prueba en la nube.
 9. **Reflexión IA** (bloque 10).
-

12. Entregables

- Repositorio Git (IaC, Helm/manifiestos de plataforma, GitOps, flujos de trabajo).
- **Documento de arquitectura** (único o carpeta `docs/`) que integre contexto, decisiones, costos, migración y datos; índice claro. Debe incluir el **reporte de costos de la implementación** descrito en la sección 5.3 (capítulo dedicado o `docs/costos-implementacion.md` enlazado desde el índice).
- Documentación: `README`, ADRs, *runbooks*, mapa CNCF (tabla en núcleo), **propuesta Microsoft Fabric** (o equivalente) en texto y diagrama, **reflexión IA**.
- **Paquete de diagramas** con índice (`README` o `docs/diagrams/`): **todos** los del **núcleo (D1–D10, D13)** según sección 8.4; en ampliación, **D11 y D12**.
- `docs/well-architected.md` (o equivalente): matriz + revisión de riesgos.

- **Modelo y reporte de costos:** volumen certificado + proyección; sensibilidad por región y clase de almacenamiento; **costo mensual explícito de lo desplegado en lab** frente a proyección del diseño (sección 5.3).
- Plan de migración nacional (oleadas, riesgos, corte, *rollback*, híbrido).
- Presentación y demo (criterios en sección 11).

13. Criterios de evaluación (rúbrica)

Criterio	Peso	Qué valora el docente
Diseño integral DevOps/CloudOps	18%	Colaboración, cambio, operación, seguridad, costo, mejora continua; reporte de costos de la implementación (sección 5.3) claro y trazable a fuentes oficiales
Arquitectura y resiliencia	13%	Multi-región o DR razonado, red territorial Colombia, oleadas de migración, RTO/RPO, <i>backup</i> frente a gravedad de datos, regiones reales citadas; arquitectura de dato (Fabric o equivalente) coherente con el inventario certificado
Well-Architected	12%	Núcleo: ≥ 2 filas/pilar y 5 riesgos; trazabilidad a evidencia o "diseño: pendiente en lab" ; ampliación: más densidad
Diagramación	10%	Núcleo D1-D10 y D13 claros (D7 con zona analítica; D8 con monitoreo operativo); D11, D12 para ampliación; leyenda "diseño solamente" donde aplique
IaC y calidad	13%	Módulos, estado remoto, pruebas estáticas, deriva
Pipelines y GitOps	12%	Pipelines separados, OIDC, <i>gates</i> , promoción, firma/SBOM si aplica
CNCF	9%	Mapa, dos novedades, ADR comparativo
Observabilidad, monitoreo y SRE	7%	SLOs, alertas, <i>runbooks</i> , monitoreo (checks, infra, híbrido) según bloque 6; carga o <i>chaos</i> según bloque 8
Comunicación y sustentación	6%	Claridad ante arquitectos y "negocio" simulado; migración nacional; reflexión IA concreta

Total: 100%. Los pesos son orientativos; el docente puede ajustarlos si lo comunica por escrito al inicio del curso.

14. Reglas generales

- **Un** hiperescalar principal, salvo multicloud **justificada** y mínima.
 - Regiones y servicios **solo** según documentación **oficial** del proveedor (nombres y enlaces citados).
 - No materializar petabytes en laboratorio; sí **estrategia**, políticas de almacenamiento, *lifecycle*, costos y *pipelines* que gobiernen la configuración.
 - **Núcleo** (Well-Architected en umbral de filas + diagramas **D1–D10 y D13**, incluidos **D7** con Fabric o equivalente y **D8** de monitoreo) = entrega **completa en mínimo** sólido. La **ampliación** suma nota y **no** exime del núcleo si está incompleto.
 - **Assessment:** reunión opcional de aclaraciones (30 min), coordinada con el curso.
 - **IA:** permitida como acelerador; **prohibido** aceptar salidas sin cotejo con documentación oficial ni sin revisión humana antes del *merge*. **Nunca** pegar secretos, tokens ni datos personales reales en herramientas de IA.
 - Excepciones por falta de acceso a nube: **solo** con autorización escrita del docente.
-

15. Mensaje final

El enunciado es **nacional y exigente en diseño**; el curso premia **prioridad**: macro bien razonado y dibujado, **migración honesta** (incluida la deuda de inventario), **monitoreo operativo** que permita dormir con un ojo abierto, **dato ordenado** (Fabric o equivalente bien argumentado) y **laboratorio** que demuestre el **núcleo** en ejecución. La excelencia es **coherencia** (Well-Architected, CNCF Landscape, GitOps, *pipelines*), no acumular recursos sin narrativa.

¡Éxitos!